

Relatorio de Auditoria de Seguranca

arsenal-open-source

Estagio: Pos-correcao · Data: 2026-08-28

Escopo e metodologia

Linguagens	python, javascript, shell, powershell
Framework	nenhum (CLI/batch determinístico, sem servidor HTTP)
Acesso a dados	sqlite3 (SQL cru, parametrizado) e better-sqlite3 (prepared statements)
Autenticacao	nenhuma (ferramenta monousuário operada pelo dono)
Frontend	HTML estático gerado por f-string em Python, sem framework e sem sessão
Deploy	artefatos em output/ versionados no git, stacks Docker Swarm publicados como documentação

Cinco categorias auditadas, cada uma mapeada para o equivalente desta stack. Achados verificados por leitura do código; varredura textual serviu apenas para levantar candidatos.

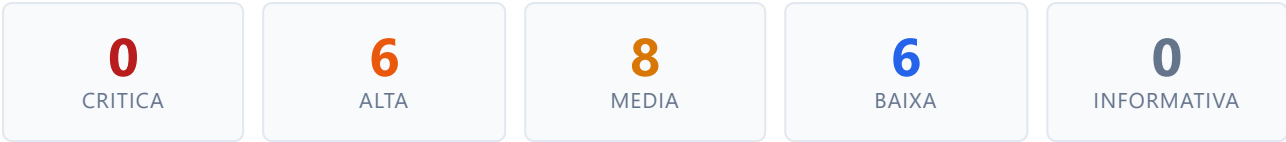
Categorias que nao se aplicam

Banco sem tranca: Não há multi-inquilino nem dono de registro. O SQLite estado_esteira.db é estado local de esteira (slugs, gates, telemetria) de um único operador, e nenhum processo remoto o consulta. Os 2 candidatos do varredor são falsos: ambos são um 'SELECT * FROM' dentro de texto de manual de troubleshooting do Haraka (scripts/data/manual-haraka.json:188 e o HTML gerado a partir dele), não uma query do projeto.

Permissao no navegador: Não existe backend, autenticação ou papel de usuário. Os HTMLs gerados são documentos estáticos sem sessão, e não há endpoint que um gate de UI pudesse esconder. Os 24 candidatos do varredor apontam todos para .claude/skills/auditoria-seguranca/scripts/varrer.py, que é o código-fonte das próprias regexes do varredor.

IDOR: Não há handler que receba identificador e devolva objeto. A única 'rota' do inventário é a string '/v1/users' dentro de modules/token-economy-core/tests/benchmark_economia.py, um literal de benchmark sem servidor associado. Sem fronteira de requisição não há referência direta a objeto a proteger.

Resumo executivo

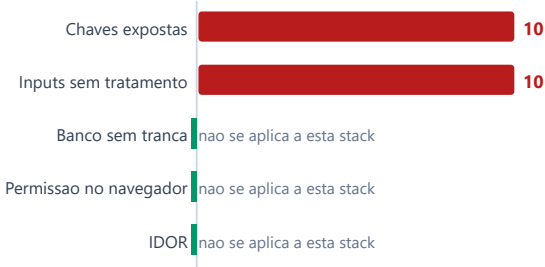


Por severidade



- Alta — 6 (30.0%)
- Media — 8 (40.0%)
- Baixa — 6 (30.0%)

Por categoria



Pontos fortes

Verificado e correto — delimita a cobertura desta auditoria.

Todas as queries SQLite são parametrizadas, nas duas linguagens. Não existe uma única f-string dentro de string SQL

```
scripts/estado_esteira.py:438 - cursor.execute("SELECT * FROM catalogo_ferramentas WHERE slug = ?", (slug,)); scripts/run_fluxo5.py:37-40 parametrizado apesar do slug vir do CLI; scripts/state-manager.js:173 - better-sqlite3 prepare() com placeholder em todas as 12 queries, e os dois db.exec() são DDL estático
```

O .env nunca foi commitado — verificado, não presumido

```
'git ls-files --error-unmatch .env' falha; 'git log --all -- .env' vem vazio; pickaxe do valor do PAT retorna 0 commits; .gitignore:19 cobre .env e .env.local. O .env.example versionado tem todas as chaves com valor vazio
```

O gerador VPS atual já usa placeholder e ensina o leitor a gerar a própria chave — é o padrão correto, e prova que o stack do ecos-google-workspace é artefato legado que sobreviveu à correção

```
scripts/modulos/vps_generator.py:738,777,810 emitem <COLE-AQUI-A-SUA-SENHA-DO-BANCO> e <COLE-AQUI-O-SEU-JWT-SECRET>; linha 1012 instrui 'gere a SUA com openssl rand -hex 32 ... Nunca reaproveite a chave de um manual - ela e publica'. Conferi os stacks dos demais alvos (ecos-crm-marketing, ecos-devops-infra, ecos-rd-station-suite, penso-suite): nenhum contém literal de senha
```

O portal da organização escapa corretamente todo dado vindo da API do GitHub, inclusive em atributo e href, e documenta o motivo

```
scripts/gerar-index-organizacao.py:69-73 aplica html.escape(..., quote=True) em name, desc, url, parent e badge antes de usá-los em data-* (76), href (79, 85) e corpo (81-84); comentário nas linhas 66-68 explica que os valores são controláveis por terceiros
```

O JavaScript embutido nos artefatos gerados usa textContent e createElement, nunca innerHTML

```
scripts/gerar-index-organizacao.py:206-213 monta o contador com createElement + textContent; grep de innerHTML/outerHTML/insertAdjacentHTML/document.write/eval/new Function sobre todos os .py retorna apenas 3 ocorrências, todas em comentários recomendando textContent
```

Validação de certificado TLS ligada por padrão na chamada ao Portainer, com válvula de escape explícita e ruidosa

```
scripts/modulos/vps_client.py:12 exige PORTAINER_TLS_INSECURE=1 para desligar e imprime aviso em stderr (linhas 18-19); verify=VERIFICAR_TLS em todas as seis chamadas HTTP
```

Nenhuma credencial ou JWT é persistida em log, JSON de estado, SQLite ou artefato de output

```
self.password e self.jwt de vps_client.py só entram no payload de /api/auth e no header Authorization; o dict devolvido por vps_auditor.audit() contém apenas hardware, redes, portas e nomes de container. Varredura do estado_esteira.db rastreado nas 11 tabelas por padrões de segredo: 0 ocorrências
```

O git_sync aborta o push quando o commit é bloqueado, em vez de publicar os commits anteriores

```
scripts/git_sync.py:71-81 - retorno diferente de zero em 'git commit' cancela o push com a justificativa 'hook que bloqueia o commit e nao impede a publicacao nao e barreira, e aviso'
```

Não há desserialização insegura nem execução por shell no código Python

Zero ocorrências de `pickle.load`, `yaml.load` sem `SafeLoader`, `eval()`, `exec()` e `shell=True` em todo o repositório fora de `node_modules/.git/output`. Todos os `subprocess.run` passam lista de argumentos, inclusive o de mensagem de commit

O expurgo automático executado pelo pre-commit é confinado e não escapa da raiz

`scripts/limpar_entulho.py:32-45` itera `os.listdir` de dois diretórios fixos, sem recursão e sem glob de usuário; `rmtree` só em nomes iniciados por `'temp_'` e remove só em `*.bak`, `*.tmp` e `repomix-*.xml`; o único `os.system` (linhas 90-92) interpola apenas `SCRIPTS_DIR`, derivado de `__file__`

O hook pre-commit encadeia o verificador de segredos como último gate e falha fechado, e o hook instalado está idêntico à fonte versionada

`.git/hooks/pre-commit` linha final chama `scripts/verificar_segredos.py` com `'|| exit 1'`; diff contra `scripts/hooks/pre-commit` é idêntico. O verificador isenta corretamente `.env.example` (`ENV_MODELO`) e bloqueia `.env`, `*.pem`, `*.key`, `id_rsa` por nome de arquivo

Achados

Severidade	Arquivo:linha	Descricao
Alta	output/05-auditorias-vps/ecossistemas/ecos-google-workspace/02-guias-de-engenharia-e-infraestrutura/01-stack-swarm-producao-integrada.yml:9, 48, 81	Stack Swarm de produção versionada com senha de banco e JWT do OnlyOffice literais, iguais para todo leitor
Alta	output/04-ecossistemas/ecos-google-workspace/02-guias-de-engenharia-e-infraestrutura/docker-stack-workspace.yml:11, 54, 89	Duplicata do stack do Fluxo 4 carrega os mesmos segredos literais
Alta	scripts/verificar_segredos.py:37, 38, 39	Gate de segredos só detecta valor entre aspas, deixando passar atribuição YAML e docker -e
Alta	scripts/compilar_35_39.py:501	Quickstart do Keycloak publica credencial de administrador admin/admin em bloco copiável
Alta	scripts/data/dossie-vertical-twilio.json:41	Comando rápido do WAHA publica chave de API literal para a ponte de WhatsApp
Alta	scripts/data/dossie-operacao-industrial.json:142	Comando rápido do Dolibarr publica senha de administrador do ERP como admin
Media	scripts/data/dossie-vertical-revops.json:41	Comando rápido do Odoo fixa a senha do Postgres em valor conhecido
Media	scripts/data/dossie-vertical-salesforce.json:163	Mesma senha de Postgres literal no dossiê de Salesforce
Media	.env:3, 8, 13, 18	Credenciais operacionais vivas em .env de texto plano na estação de trabalho
Media	scripts/compilar_compendio_diamante.py:329, 406, 358, 383, 402, 418, 445, 504	Compilador de compêndio interpola JSON cru em HTML e em href, sem nenhum escape
Media	scripts/compilar_compendio_vertical.py:388, 380, 384	Compilador de dossiê vertical repete a interpolação sem escape em href e corpo

Severidade	Arquivo:linha	Descrição
Media	scripts/compilar_ecossistema_tripartite.py:902, 1007, 2450	Compilador de ecossistema monta href com aspas simples, tornando a quebra de atributo trivial
Media	scripts/padrees/template_trilha_aprendizado.py:242, 249, 251, 252, 254, 257, 258, 259	Bibliografia de trilha injeta URL, título e autor de pesquisa web direto no HTML
Media	scripts/padrees/template_manual_operacional.py:426, 427, 428, 429, 430	Tabela de fontes verificadas do manual operacional interpola dados de terceiros sem escape
Baixa	scripts/data/manual-mautic.json:163	Campo SMTP_PASSWORD escapa do padrão de placeholder que o próprio manual ensina a substituir
Baixa	scripts/modulos/vps_generator.py:1227, 1228, 1231, 1232	Sanitização parcial: vps_generator escapa a tabela e esquece o target-box
Baixa	scripts/renderizador_html_diamante.py:176, 177, 178, 179, 199, 200, 201, 202, 203	Renderizador desfaz o escape do Pandoc em blocos Mermaid e ainda liga securityLevel loose
Baixa	scripts/linter.js:24, 29, 32	linter.js executa eslint via shell com lista de arquivos concatenada sem quoting
Baixa	scripts/run_fluxo5.py:109, 110, 111, 112, 113, 114, 116	Slug de CLI vira caminho de escrita sem allowlist, e componente absoluto descarta a base
Baixa	scripts/compilar_lista_horizontal_tripartite.py:131, 136, 137, 138, 152, 153, 154, 155, 156	Slug do Fluxo 1 entra em caminho sem remover separadores nem sequência de subida

Chaves expostas

Alta

output/05-auditorias-vps/ecossistemas/ecos-google-workspace/02-guias-de-engenharia-e-infraestrutura/01-stack-swarm-producao-integrada.yml:9, 48, 81

Stack Swarm de produção versionada com senha de banco e JWT do OnlyOffice literais, iguais para todo leitor

```
POSTGRES_PASSWORD: ****  
POSTGRES_PASSWORD: ****  
JWT_SECRET: ****
```

Por que e explorável: O arquivo é rastreado pelo git (git ls-files confirma) e é o artefato que o leitor aplica com 'docker stack deploy'. Os valores são literais fixos, não placeholders: todo mundo que baixar o repositório e seguir o playbook sobe Postgres e OnlyOffice com exatamente a mesma senha e o mesmo segredo de assinatura. O JWT_SECRET do OnlyOffice autoriza requisições ao document server — quem o conhece forja comandos de abertura e conversão de documentos. Os labels Traefik do mesmo arquivo apontam para domínios reais em produção (drive./office.vpsconexao.org), então o alvo não é hipotético.

Impacto: Acesso ao banco Postgres do Nextcloud (dados de arquivos e usuários) e forja de requisições ao OnlyOffice Document Server em qualquer instalação feita a partir deste playbook, incluindo a do próprio autor se a stack foi aplicada.

Condições: Basta ler o repositório. Torna-se crítica se esta stack estiver de fato aplicada na VPS do autor — não foi possível verificar remotamente.

Correção: Regenerar o alvo ecos-google-workspace com o gerador atual (scripts/modulos/vps_generator.py já emite <COLE-AQUI-...> + instrução 'openssl rand -hex 32' na linha 1012) e substituir os literais por placeholders. Se a stack foi aplicada, rotacionar a senha do Postgres e o JWT do OnlyOffice na VPS e purgar os valores do histórico git.

Alta

output/04-ecossistemas/ecos-google-workspace/02-guias-de-engenharia-e-infraestrutura/docker-stack-workspace.yml:11, 54, 89

Duplicata do stack do Fluxo 4 carrega os mesmos segredos literais

```
POSTGRES_PASSWORD: ****  
POSTGRES_PASSWORD: ****  
JWT_SECRET: ****
```

Por que e explorável: Mesmo conteúdo do achado anterior, em segundo caminho versionado (Fluxo 4). Corrigir só um dos dois deixa o segredo disponível no outro, e o RELATORIO-AUDITORIA-E-INTEGRACAO-VPS.md do mesmo diretório repete os valores no corpo do texto. Os mesmos literais também aparecem em LIVRO-AUDITORIA-E-INCORPORACAO-VPS.html/.md e no ecos-google-workspace-vps.typ.

Impacto: Idêntico ao achado do stack do Fluxo 5, com superfície ampliada: seis arquivos versionados carregam o mesmo par de segredos.

Condições: Basta ler o repositório.

Correção: Regenerar os seis artefatos derivados do alvo ecos-google-workspace de uma vez, e adicionar um gate mecânico que recuse qualquer .yml em output/ com valor literal em campo *PASSWORD*/*SECRET*.

Alta

scripts/verificar_segredos.py:37, 38, 39

Gate de segredos só detecta valor entre aspas, deixando passar atribuição YAML e docker -e

```
("senha atribuida", re.compile(
    r'(?i)[\w.-]*(password|passwd|senha|secret|api[_-]?key|access[_-]?token|token)'
    r'\s*[:=]\s*["\'][^"\'$\{\}[\^"\']{7,}["\']'))
```

Por que e explorável: O padrão heurístico exige que o valor esteja delimitado por aspas. As duas formas mais comuns de segredo em um repositório de infraestrutura não têm aspas: 'POSTGRES_PASSWORD: senha' em YAML e 'docker run -e ADMIN_PASSWORD=senha' em shell. Verifiquei executando o próprio gate contra os arquivos que comprovadamente contêm segredo literal: exit 0 em 01-stack-swarm-producao-integrada.yml, em scripts/compilar_35_39.py e em scripts/data/dossie-vertical-twilio.json. O Gate 1 do .git/hooks/pre-commit não cobre a lacuna: sua lista tem apenas padrões de formato forte (ghp_, sk-, AKIA, Alza, xox, PEM), nenhum padrão genérico de senha. É por isso que os segredos dos dois achados anteriores estão versionados.

Impacto: Todo segredo que não tenha formato de token reconhecível entra no repositório sem resistência, e a barreira dá a impressão de estar cobrindo o caso.

Condições: Nenhuma. Reproduzível com 'python scripts/verificar_segredos.py --arquivo <qualquer um dos três>'.

Correção: Tornar as aspas opcionais no PADROES_FRACOS (aceitar valor não delimitado até espaço/quebra) e adicionar teste de regressão que exija exit 1 para 'CHAVE: valor' em YAML e '-e CHAVE=valor' em shell. Espelhar a mesma regra no Gate 1 do hook.

Alta

scripts/compilar_35_39.py:501

Quickstart do Keycloak publica credencial de administrador admin/admin em bloco copiável

```
"quickstart": "docker run -p 8080:8080 -e KEYCLOAK_ADMIN=**** -e KEYCLOAK_ADMIN_PASSWORD=****
quay.io/keycloak/keycloak:latest start-dev"
```

Por que e explorável: O valor é materializado em output/01-listas-horizontais/list-iam-zero-trust-sso/materiais/list-iam-zero-trust-sso.html:472 dentro de um .code-block com botão 'Copiar', sem nenhum aviso adjacente de que 'start-dev' é modo de desenvolvimento nem de que a credencial precisa ser trocada. O público-alvo declarado do projeto (regra R21: advogados, médicos, gestores) não tem como inferir isso. Keycloak é o provedor de identidade que federa os demais sistemas: quem entra no console de admin assume todos os realms e todas as aplicações por trás do SSO.

Impacto: Comprometimento total da identidade corporativa de qualquer leitor que siga o comando — sequestro de sessão em todas as aplicações federadas.

Condições: O leitor precisa expor a porta 8080. O comando publicado já faz isso ('-p 8080:8080').

Correção: Substituir por variável com instrução de geração ('-e KEYCLOAK_ADMIN_PASSWORD=\$(openssl rand -base64 24)') e acrescentar aviso explícito de que 'start-dev' não é modo de produção. Regenerar a lista list-iam-zero-trust-sso.

Alta

scripts/data/dossie-vertical-twilio.json:41

Comando rápido do WAHA publica chave de API literal para a ponte de WhatsApp

```
"comando_rapido": "docker run -d -p 3000:3000 --name waha -e WHATSAPP_API_KEY=**** -v waha_sessions:/app/.sessions devlikeapro/waha:latest"
```

Por que e explorável: A WHATSAPP_API_KEY é o único controle de acesso da API do WAHA. O comando publica a porta 3000 e fixa a chave num valor literal que o dossiê afirma ser 'autenticação segura via API Key'. O valor chega ao leitor em output/02-dossies-verticais/vert-twilio/materiais/vert-twilio.html e .md como bloco copiável. Quem conhece a chave envia e lê mensagens na sessão de WhatsApp do leitor e baixa o volume waha_sessions.

Impacto: Controle da conta de WhatsApp conectada: envio de mensagens em nome do leitor, leitura do histórico e captura da sessão autenticada.

Condições: A porta 3000 precisa estar alcançável. O comando publicado a expõe.

Correção: Trocar por '-e WHATSAPP_API_KEY=\$(openssl rand -hex 32)' com instrução de anotar o valor, e regenerar vert-twilio.

Alta

scripts/data/dossie-operacao-industrial.json:142

Comando rápido do Dolibarr publica senha de administrador do ERP como admin

```
"comando_rapido": "docker run -d -p 80:80 -e DOLI_ADMIN_PASSWORD=**** --name doli tuxgasy/dolibarr"
```

Por que e explorável: Publica o ERP na porta 80 com senha de administrador literal e trivial, sem aviso. Materializado em output/01-listas-horizontais/list-operacao-industrial/materiais/list-operacao-industrial.html como bloco copiável. Dolibarr concentra faturamento, propostas, estoque e cadastro de clientes.

Impacto: Acesso administrativo aos dados financeiros e à base de clientes de qualquer leitor que siga o comando.

Condições: Nenhuma além de o leitor executar o comando publicado, que já expõe a porta 80.

Correção: Substituir por geração de senha no ato e regenerar a lista list-operacao-industrial.

Media

scripts/data/dossie-vertical-revops.json:41

Comando rápido do Odoo fixa a senha do Postgres em valor conhecido

```
"comando_rapido": "docker run -d -p 8069:8069 --name odoo-revops -e POSTGRES_PASSWORD=**** --link db:db -t odoo:latest"
```

Por que e explorável: Senha de banco literal em comando copiável, materializada em output/02-dossies-verticais/vert-revops/materiais/vert-revops.html e em list-crm-erp-corporativo.html. O impacto é menor que os anteriores porque o Postgres fica atrás de '--link' e não é publicado diretamente, mas a senha vira conhecida para todo leitor e basta um segundo container na mesma rede ou uma exposição posterior da porta 5432 para valer.

Impacto: Acesso ao banco do Odoo (dados de vendas, faturamento e clientes) por quem já tenha alcance à rede do container.

Condicoes: Exige alcance à rede Docker do leitor ou publicação posterior da porta do Postgres.

Correcao: Gerar a senha no comando e regenerar vert-revops e list-crm-erp-corporativo.

Media

scripts/data/dossie-vertical-salesforce.json:163

Mesma senha de Postgres literal no dossiê de Salesforce

```
"comando_rapido": "docker run -d -p 8069:8069 --name odoo -e POSTGRES_PASSWORD=**** --link db:db -t odoo:latest"
```

Por que e explorável: Repetição do mesmo defeito em outro dossiê (materializado em output/02-dossies-verticais/vert-salesforce/materiais/vert-salesforce.html). Registrado à parte porque a correção do arquivo anterior não o alcança — os JSONs de dados são independentes.

Impacto: Idêntico ao achado de revops.

Condicoes: Exige alcance à rede Docker do leitor.

Correcao: Gerar a senha no comando e regenerar vert-salesforce.

Media

.env:3, 8, 13, 18

Credenciais operacionais vivas em .env de texto plano na estação de trabalho

```
GITHUB_TOKEN="****"  
PORTAINER_PASSWORD="****"  
VPS_PASSWORD="****"  
MAUTIC_PASSWORD="****"
```

Por que e explorável: Verifiquei antes de classificar, sem presumir: 'git ls-files --error-unmatch .env' falha (não rastreado), 'git log --all -- .env' vem vazio, o pickaxe do valor do PAT retorna 0 commits, e .gitignore:19 o cobre. Não é vazamento: é superfície local. O que sobra de real é que o arquivo guarda em texto plano a senha de root da VPS, a senha do painel Portainer, um PAT do GitHub e a senha do Mautic, sem cifra e sem escopo — e qualquer processo, script ou subagente executado nesta pasta lê os quatro de uma vez (eu mesmo li). O grep pelos valores confirma que nenhum deles aparece em qualquer outro arquivo do repositório.

Impacto: Um único processo hostil ou subagente mal direcionado nesta estação obtém root na VPS, controle do orquestrador de containers, escrita no repositório e a conta de e-mail marketing, sem precisar de escalada.

Condições: Exige execução de código na estação de trabalho do dono. Sem vetor remoto.

Correção: Nada a corrigir no repositório — o arquivo está corretamente ignorado. Decisão do dono: trocar a senha de root da VPS por chave SSH, reduzir o PAT do GitHub ao escopo mínimo com validade curta, e considerar um cofre em vez de texto plano. Rotação não é exigida por vazamento, e sim por higiene de credencial de longa duração.

Baixa

scripts/data/manual-mautic.json:163

Campo SMTP_PASSWORD escapa do padrão de placeholder que o próprio manual ensina a substituir

```
SMTP_USER=seu-email@seuservidor.com.br\nSMTP_PASSWORD=****\nSMTP_ENCRYPTION=tls
```

Por que e explorável: No mesmo bloco, DB_PASSWORD e APP_SECRET usam a forma <COLE-AQUI-...> e o texto seguinte instrui: 'Repare que no arquivo acima existem campos escritos assim: <COLE-AQUI-...>. Eles não são senhas de verdade.' O SMTP_PASSWORD não segue esse padrão e carrega um literal, então a instrução não o cobre. Um leitor não-técnico seguindo o manual à risca não é levado a trocá-lo. O valor em si não autentica em lugar nenhum, então não há credencial exposta — o defeito é a inconsistência que ensina o leitor a confiar num critério visual incompleto.

Impacto: Configuração de e-mail quebrada e, pior, erosão da regra de leitura que o próprio manual estabelece para identificar o que precisa ser substituído.

Condições: Depende do leitor seguir literalmente o critério <COLE-AQUI-...> que o manual define.

Correção: Trocar por <COLE-AQUI-A-SUA-SENHA-DO-EMAIL> para uniformizar o padrão, e regenerar o manual do Mautic.

Inputs sem tratamento

Media

scripts/compilar_compendio_diamante.py:329, 406, 358, 383, 402, 418, 445, 504

Compilador de compêndio interpola JSON cru em HTML e em href, sem nenhum escape

```
repo_url = f.get("repositorio_github", "#")
...
<a class="repo-btn" href="{repo_url}" rel="noopener noreferrer" target="_blank">
```

Por que e explorável: O arquivo tem zero chamadas a `html.escape` (contei: 0) e monta todo o markup por f-string. O campo `repositorio_github` entra direto num atributo `href` delimitado por aspas duplas: um valor com aspas quebra o atributo e injeta manipulador de evento, e um valor `'javascript:...'` produz link executável. Os campos de texto (nome, veredito, o_que_faz) entram crus no corpo. A origem não é constante de código: os JSONs de `scripts/data` são preenchidos por pesquisa web da esteira (URLs de repositório, descrições e títulos coletados de terceiros) e o arquivo de dados é escolhido por argumento de CLI.

Impacto: XSS armazenado no HTML distribuído ao leitor final — execução de script no contexto do documento aberto por advogados, médicos e gestores, que é o público declarado.

Condições: Exige que um campo do JSON de dados carregue conteúdo hostil, o que é plausível porque esses campos vêm de coleta web.

Correção: Adicionar helper `_esc` com `html.escape(..., quote=True)` aplicado a toda interpolação de campo do JSON, e validar `repositorio_github` contra allowlist de esquema (`^https?:/`).

Media

scripts/compilar_compendio_vertical.py:388, 380, 384

Compilador de dossiê vertical repete a interpolação sem escape em href e corpo

```
<a class="repo-btn" href="{repo_url}" rel="noopener noreferrer" target="_blank">
```

Por que e explorável: Mesmo padrão do compilador de compêndio, também com zero chamadas de escape no arquivo inteiro. Consome `scripts/data/dossie-vertical-*.json`, cujos campos de URL e descrição vêm de coleta externa.

Impacto: XSS armazenado nos dossiês verticais distribuídos.

Condições: Campo hostil no JSON de dossiê.

Correção: Mesmo helper de escape e mesma allowlist de esquema de URL.

Media

scripts/compilar_ecossistema_tripartite.py:902, 1007, 2450

Compilador de ecossistema monta href com aspas simples, tornando a quebra de atributo trivial

```
linhas_tab += f"<tr>...<td><a href='{f_item['repositorio_github']}' target='_blank' style='...'>GitHub</a></td></tr>"
```

Por que e explorável: Zero chamadas de escape no arquivo. A linha 2450 é a pior das três porque o atributo é delimitado por aspas simples: basta um caractere ' no valor para fechar o atributo e injetar onmouseover=. Na mesma linha, nome, saas_substituido_direto e economia_anual_str também entram crus. As linhas 902 e 1007 repetem o href sem escape com aspas duplas.

Impacto: XSS armazenado nos dossiês de macro-ecossistema.

Condições: Campo hostil no JSON de ecossistema.

Correção: Escapar com quote=True e padronizar aspas duplas nos atributos.

Media

scripts/padroles/template_trilha_aprendizado.py:242, 249, 251, 252, 254, 257, 258, 259

Bibliografia de trilha injeta URL, título e autor de pesquisa web direto no HTML

```
<a href="{r['url']}" target="_blank" rel="noopener noreferrer">{r['titulo']}</a>
```

Por que e explorável: Zero chamadas de escape no arquivo. Este é o caso de origem externa mais direto do repositório: rastreei scripts/gerar_trilha_aprendizado.py:83-89, que lê scripts/data/trilha-<slug>.json e repassa o dicionário sem tratamento; inspecionei scripts/data/trilha-screenpipe.json e os registros de recurso são fontes web reais (url https://dev.to/..., título, autor, aprendizado_chave em texto livre). Ou seja, o campo que vai para o href é exatamente uma URL de terceiro coletada pela esteira, e o esquema não é validado — 'javascript:' passa.

Impacto: XSS armazenado nas trilhas de aprendizado entregues ao leitor final.

Condições: Exige que a coleta de fontes traga um título, autor ou URL com conteúdo hostil.

Correção: Escapar todos os campos de recurso com quote=True e rejeitar URL cujo esquema não seja http/https.

Media

scripts/padrees/template_manual_operacional.py:426, 427, 428, 429, 430

Tabela de fontes verificadas do manual operacional interpola dados de terceiros sem escape

```
<td>{r['autor_ou_canal']}</td>
<td><a href="{r['url']}" target="_blank" rel="noopener noreferrer" style="...">Acessar Fonte Original</a></td>
```

Por que e explorável: Zero chamadas de escape. Rastreei a origem em scripts/gerar_manual_operacional.py:217-230: os registros vêm de scripts/data/manual-<slug>.json, seção de fontes verídicas, que a própria documentação do Fluxo 3 descreve como 'auditoria de 5 fontes verificadas' coletadas da web. Título, autor e URL são conteúdo de terceiro entrando em corpo e em href sem tratamento.

Impacto: XSS armazenado nos manuais operacionais de VPS, que são o material com maior circulação do projeto.

Condições: Fonte coletada com conteúdo hostil.

Correção: Escapar os campos de fonte com quote=True e validar o esquema da URL.

Baixa

scripts/modulos/vps_generator.py:1227, 1228, 1231, 1232

Sanitização parcial: vps_generator escapa a tabela e esquece o target-box

```
target_name = prof.get('name', hero_title)
role_name = prof.get('role', 'Stack e Módulos de Produção')
...
<h4>{target_name}</h4>
<p>{role_name} · Garantia de Risco Zero...</p>
```

Por que e explorável: O arquivo define o helper correto _esc (linhas 6-15, com html.escape quote=True) e o aplica em nove pontos (1142, 1147, 1159, 1163, 1167, 1174, 1189, 1194), mas não nestes dois. É a assinatura clássica de sanitização parcial: o mesmo valor passa escapado quando entra pela tabela markdown e cru quando entra pelo target-box. A origem é o slug de CLI: scripts/modulos/vps_decision_engine.py:319-336 devolve name=f'Ferramenta {tool_slug}' para qualquer slug não cadastrado — que é o caminho normal de qualquer alvo novo — propagando o argumento de --ferramenta até o <h4>.

Impacto: Injeção de markup no HTML de auditoria VPS. Explorabilidade contida porque quem passa o slug é o operador ou o agente que ele dirige, não um terceiro remoto.

Condições: Requer que o slug chegue de fonte não confiável — plausível neste projeto, onde os fluxos são acionados por comandos de agente ('/fluxo5 [slug]') sujeitos a injeção de prompt, mas não é vetor direto.

Correção: Envolver target_name e role_name no _esc que já existe no arquivo. Correção de duas linhas.

Baixa

scripts/renderizador_html_diamante.py:176, 177, 178, 179, 199, 200, 201, 202, 203

Renderizador desfaz o escape do Pandoc em blocos Mermaid e ainda liga securityLevel loose

```
conteudo = conteudo.replace("&", "&").replace(""", "'').replace("<", "< "
<").replace(">", ">")
return f'<pre class="mermaid">\n{conteudo.strip()}\n</pre>'
...
mermaid.initialize({{ startOnLoad: true, theme: 'neutral', securityLevel: 'loose' }});
```

Por que e explorável: O Pandoc já havia escapado o bloco de código; este replace reverte a sanitização e reinsere < e > crus no documento, de modo que um bloco ```mermaid` contendo `</pre> <script>` escapa do `<pre>`. securityLevel 'loose' agrava: o Mermaid passa a aceitar HTML em labels e callbacks click. A origem é o conteúdo de arquivos .md lidos do disco (verifiquei scripts/compilar_playbook_fluxo1.py:53-65, que concatena todos os .md de especificacoes/), hoje escritos pelo próprio projeto — por isso baixa, e não média.

Impacto: Execução de script no playbook HTML se um .md de especificação vier de fora ou for editado por terceiro.

Condições: Depende de um .md de origem não confiável entrar em especificacoes/. Hoje todos são do próprio autor.

Correção: Trocar securityLevel para 'strict' e, em vez de desescapar o bloco inteiro, reverter só as entidades dentro do texto do diagrama após validar que ele não contém '<'.

Baixa

scripts/linter.js:24, 29, 32

linter.js executa eslint via shell com lista de arquivos concatenada sem quoting

```
const args = ["eslint", "--format=json", fix ? "--fix" : "", ...globs].filter(Boolean);
try {
  const output = execSync(args.join(" "), {
```

Por que e explorável: execSync recebendo string roda através do shell. globs vem de process.argv ou de stdin separado por linha, sem quoting nem escape, então um caminho contendo ;, &&, crase ou \$() executa comando arbitrário. Verifiquei os chamadores: nenhum script do repositório invoca linter.js — as únicas referências são documentação em relatórios. Sem alimentador automatizado, o vetor depende de alguém canalizar uma lista de nomes de arquivo de origem não confiável para o stdin.

Impacto: Execução de comando arbitrário na estação de quem rodar o linter com entrada hostil.

Condições: Exige alimentar o stdin com nomes de arquivo controlados por terceiro. Não há pipeline atual que faça isso.

Correção: Trocar por execFileSync("eslint", args), que não passa pelo shell.

Baixa

scripts/run_fluxo5.py:109, 110, 111, 112, 113, 114, 116

Slug de CLI vira caminho de escrita sem allowlist, e componente absoluto descarta a base

```
out_dir = os.path.join(root_vps_output, "ecossistemas", t_slug)
...
out_dir = os.path.join(root_vps_output, "ferramentas", t_slug)
gen = VPSGenerator(audit_data, decision, out_dir, base_domain=args.base_domain)
```

Por que e explorável: `t_slug` vem de `--ecossistema/--ferramenta/--ecossistemas/--ferramentas` sem normalização nem allowlist, e `scripts/modulos/vps_decision_engine.py:319-336` aceita slug desconhecido em vez de recusar. Duas propriedades combinam: `os.path.join` descarta a base quando o último componente é absoluto, então `--ferramenta 'C:\Windows\Temp\x'` escreve fora de output; e `'../..'` também escapa. O destino é criado em `vps_generator.py:184` e recebe cerca de vinte `open(..., 'w')` de nomes fixos.

Impacto: Criação de diretório e sobrescrita de arquivo fora da raiz do projeto, limitada aos nomes de arquivo fixos do gerador.

Condições: Quem passa o slug é o operador, que já pode escrever onde quiser — não há fronteira de privilégio cruzada. Relevante apenas porque os fluxos são acionados por agente (`'/fluxo5 [slug]'`), o que abre a possibilidade de o valor vir de injeção de prompt.

Correção: Validar o slug contra `^[a-z0-9-]{1,35}$` (a própria regra R13 já exige isso) e recusar o resto, em vez de cair em perfil genérico.

Baixa

scripts/compilar_lista_horizontal_tripartite.py:131, 136, 137, 138, 152, 153, 154, 155, 156

Slug do Fluxo 1 entra em caminho sem remover separadores nem sequência de subida

```
slug_limpo = slug_ou_json.replace("list-", "").replace(".html", "").replace(".json", "")
...
out_bundle = BASE_DIR / "output" / "01-listas-horizontais" / f"list-{slug_limpo}"
out_materiais.mkdir(parents=True, exist_ok=True)
```

Por que e explorável: A 'limpeza' remove apenas três literais e não filtra `/ \` nem `'..'`, então `--slug '../..../tmp/x'` é segmentado pelo `pathlib` e escapa de output/, com `mkdir(parents=True)` criando a árvore e `write_text` gravando. As linhas 136-138 usam o mesmo argumento como caminho de leitura direto, inclusive absoluto, permitindo carregar JSON de fora da raiz.

Impacto: Escrita e leitura de arquivo fora do diretório do projeto a partir de um argumento de CLI.

Condições: Mesma fronteira do achado anterior: o operador, ou o agente que ele dirige, controla o argumento.

Correção: Aplicar a validação de slug da regra R13 antes de compor qualquer caminho e resolver o resultado com `Path.resolve()`, recusando o que não estiver sob `BASE_DIR`.

| Issues para o GitHub

Cada bloco abaixo e uma issue completa, pronta para copiar.

--- ISSUE 1 ---

[Seguranca] Stack Swarm de produção versionada com senha de banco e JWT do OnlyOffice literais, iguais para todo leitor

****Labels:**** `security`, `alta`

****Categoria:**** Chaves expostas

Problema

O arquivo é rastreado pelo git (git ls-files confirma) e é o artefato que o leitor aplica com 'docker stack deploy'. Os valores são literais fixos, não placeholders: todo mundo que baixar o repositório e seguir o playbook sobe Postgres e OnlyOffice com exatamente a mesma senha e o mesmo segredo de assinatura. O JWT_SECRET do OnlyOffice autoriza requisições ao document server – quem o conhece forja comandos de abertura e conversão de documentos. Os labels Traefik do mesmo arquivo apontam para domínios reais em produção (drive./office.vpsconexao.org), então o alvo não é hipotético.

Evidencia

`output/05-auditorias-vps/ecossistemas/ecos-google-workspace/02-guias-de-engenharia-e-infraestrutura/01-stack-swarm-producao-integrada.yml:9, 48, 81`

...

POSTGRES_PASSWORD: ****

POSTGRES_PASSWORD: ****

JWT_SECRET: ****

...

Impacto

Acesso ao banco Postgres do Nextcloud (dados de arquivos e usuários) e forja de requisições ao OnlyOffice Document Server em qualquer instalação feita a partir deste playbook, incluindo a do próprio autor se a stack foi aplicada.

****Condicoes:**** Basta ler o repositório. Torna-se crítica se esta stack estiver de fato aplicada na VPS do autor – não foi possível verificar remotamente.

Correcao sugerida

Regenerar o alvo ecos-google-workspace com o gerador atual (scripts/modulos/vps_generator.py já emite <COLE-AQUI-...> + instrução 'openssl rand -hex 32' na linha 1012) e substituir os literais por placeholders. Se a stack foi aplicada, rotacionar a senha do Postgres e o JWT do OnlyOffice na VPS e purgar os valores do histórico git.

Criterios de aceite

- [] Correcao aplicada em `output/05-auditorias-vps/ecossistemas/ecos-google-workspace/02-guias-de-engenharia-e-infraestrutura/01-stack-swarm-producao-integrada.yml`
- [] Teste automatizado que falha antes e passa depois
- [] Nova varredura nao reporta este achado
- [] Credencial rotacionada no provedor (o valor antigo esta comprometido)

--- FIM ISSUE 1 ---

--- ISSUE 2 ---

[Seguranca] Duplicata do stack do Fluxo 4 carrega os mesmos segredos literais

****Labels:**** `security`, `alta`

****Categoria:**** Chaves expostas

Problema

Mesmo conteúdo do achado anterior, em segundo caminho versionado (Fluxo 4). Corrigir só um dos dois deixa o segredo disponível no outro, e o RELATORIO-AUDITORIA-E-INTEGRACAO-VPS.md do mesmo diretório repete os valores no corpo do texto. Os mesmos literais também aparecem em LIVRO-AUDITORIA-E-INCORPORACAO-VPS.html/.md e no ecos-google-workspace-vps.typ.

Evidencia

`output/04-ecossistemas/ecos-google-workspace/02-guias-de-engenharia-e-infraestrutura/docker-stack-workspace.yml:11, 54, 89`

```

POSTGRES\_PASSWORD: \*\*\*\*

POSTGRES\_PASSWORD: \*\*\*\*

JWT\_SECRET: \*\*\*\*

```

Impacto

Idêntico ao achado do stack do Fluxo 5, com superfície ampliada: seis arquivos versionados carregam o mesmo par de segredos.

****Condições:**** Basta ler o repositório.

Correcao sugerida

Regenerar os seis artefatos derivados do alvo ecos-google-workspace de uma vez, e adicionar um gate mecânico que recuse qualquer .yml em output/ com valor literal em campo *PASSWORD*/*SECRET*.

Criterios de aceite

- [] Correcao aplicada em `output/04-ecossistemas/ecos-google-workspace/02-guias-de-engenharia-e-infraestrutura/docker-stack-workspace.yml`
- [] Teste automatizado que falha antes e passa depois
- [] Nova varredura nao reporta este achado
- [] Credencial rotacionada no provedor (o valor antigo esta comprometido)

--- FIM ISSUE 2 ---

--- ISSUE 3 ---

[Seguranca] Gate de segredos só detecta valor entre aspas, deixando passar atribuição YAML e docker -e

****Labels:**** `security`, `alta`
****Categoria:**** Chaves expostas

Problema

O padrão heurístico exige que o valor esteja delimitado por aspas. As duas formas mais comuns de segredo em um repositório de infraestrutura não têm aspas: 'POSTGRES_PASSWORD: senha' em YAML e 'docker run -e ADMIN_PASSWORD=senha' em shell. Verifiquei executando o próprio gate contra os arquivos que comprovadamente contêm segredo literal: exit 0 em 01-stack-swarm-producao-integrada.yml, em scripts/compilar_35_39.py e em scripts/data/dossie-vertical-twilio.json. O Gate 1 do .git/hooks/pre-commit não cobre a lacuna: sua lista tem apenas padrões de formato forte (ghp_, sk-, AKIA, AIza, xox, PEM), nenhum padrão genérico de senha. É por isso que os segredos dos dois achados anteriores estão versionados.

Evidencia

`scripts/verificar\_segredos.py:37, 38, 39`

...

```
("senha atribuida", re.compile(
    r'(?i)[\w.-]*(password|passwd|senha|secret|api[_-]?key|access[_-]?token|token)'
    r'\s*[:=]\s*"\'[^\s$\'"]{7,}["\']'))
...
```

Impacto

Todo segredo que não tenha formato de token reconhecível entra no repositório sem resistência, e a barreira dá a impressão de estar cobrindo o caso.

****Condicoes:**** Nenhuma. Reproduzível com 'python scripts/verificar_segredos.py --arquivo <qualquer um dos três>'.

Correcao sugerida

Tornar as aspas opcionais no PADROES_FRACOS (aceitar valor não delimitado até espaço/quebra) e adicionar teste de regressão que exija exit 1 para 'CHAVE: valor' em YAML e '-e CHAVE=valor' em shell. Espelhar a mesma regra no Gate 1 do hook.

Criterios de aceite

- [] Correcao aplicada em `scripts/verificar\_segredos.py`
 - [] Teste automatizado que falha antes e passa depois
 - [] Nova varredura nao reporta este achado
 - [] Credencial rotacionada no provedor (o valor antigo esta comprometido)
- FIM ISSUE 3 ---

--- ISSUE 4 ---

[Seguranca] Quickstart do Keycloak publica credencial de administrador admin/admin em bloco copiável

****Labels:**** `security`, `alta`

****Categoria:**** Chaves expostas

Problema

O valor é materializado em output/01-listas-horizontais/list-iam-zero-trust-sso/materiais/list-iam-zero-trust-sso.html:472 dentro de um .code-block com botão 'Copiar', sem nenhum aviso adjacente de que 'start-dev' é modo de desenvolvimento nem de que a credencial precisa ser trocada. O público-alvo declarado do projeto (regra R21: advogados, médicos, gestores) não tem como inferir isso. Keycloak é o provedor de identidade que federa os demais sistemas: quem entra no console de admin assume todos os realms e todas as aplicações por trás do SSO.

Evidencia

```
`scripts/compilar_35_39.py:501`
```

```
...
```

```
"quickstart": "docker run -p 8080:8080 -e KEYCLOAK_ADMIN=**** -e KEYCLOAK_ADMIN_PASSWORD=****  
quay.io/keycloak/keycloak:latest start-dev"
```

```
...
```

Impacto

Comprometimento total da identidade corporativa de qualquer leitor que siga o comando – sequestro de sessão em todas as aplicações federadas.

****Condições:**** O leitor precisa expor a porta 8080. O comando publicado já faz isso ('-p 8080:8080').

Correcao sugerida

Substituir por variável com instrução de geração ('-e KEYCLOAK_ADMIN_PASSWORD=\$(openssl rand -base64 24)') e acrescentar aviso explícito de que 'start-dev' não é modo de produção. Regenerar a lista list-iam-zero-trust-sso.

Criterios de aceite

- [] Correcao aplicada em `scripts/compilar\_35\_39.py`
- [] Teste automatizado que falha antes e passa depois
- [] Nova varredura nao reporta este achado
- [] Credencial rotacionada no provedor (o valor antigo esta comprometido)

--- FIM ISSUE 4 ---

--- ISSUE 5 ---

[Seguranca] Comando rápido do WAHA publica chave de API literal para a ponte de WhatsApp

****Labels:**** `security`, `alta`

****Categoria:**** Chaves expostas

Problema

A WHATSAPP_API_KEY é o único controle de acesso da API do WAHA. O comando publica a porta 3000 e fixa a chave num valor literal que o dossiê afirma ser 'autenticação segura via API Key'. O valor chega ao leitor em output/02-dossies-verticais/vert-twilio/materiais/vert-twilio.html e .md como bloco copiável. Quem conhece a chave envia e lê mensagens na sessão de WhatsApp do leitor e baixa o volume waha_sessions.

Evidencia

`scripts/data/dossie-vertical-twilio.json:41`

```

"comando\_rapido": "docker run -d -p 3000:3000 --name waha -e WHATSAPP\_API\_KEY=\*\*\*\* -v waha\_sessions:/app/.sessions devlikeapro/waha:latest"

```

Impacto

Controle da conta de WhatsApp conectada: envio de mensagens em nome do leitor, leitura do histórico e captura da sessão autenticada.

****Condicoes:**** A porta 3000 precisa estar alcançável. O comando publicado a expõe.

Correcao sugerida

Trocar por '-e WHATSAPP_API_KEY=\$(openssl rand -hex 32)' com instrução de anotar o valor, e regenerar vert-twilio.

Criterios de aceite

- [] Correcao aplicada em `scripts/data/dossie-vertical-twilio.json`
- [] Teste automatizado que falha antes e passa depois
- [] Nova varredura nao reporta este achado
- [] Credencial rotacionada no provedor (o valor antigo esta comprometido)

--- FIM ISSUE 5 ---

--- ISSUE 6 ---

[Seguranca] Comando rápido do Dolibarr publica senha de administrador do ERP como admin

****Labels:**** `security`, `alta`

****Categoria:**** Chaves expostas

Problema

Publica o ERP na porta 80 com senha de administrador literal e trivial, sem aviso. Materializado em output/01-listas-horizontais/list-operacao-industrial/materiais/list-operacao-industrial.html como bloco copiável. Dolibarr concentra faturamento, propostas, estoque e cadastro de clientes.

Evidencia

`scripts/data/dossie-operacao-industrial.json:142`

```

"comando\_rapido": "docker run -d -p 80:80 -e DOLI\_ADMIN\_PASSWORD=\*\*\*\* --name doli  
tuxgasy/dolibarr"

```

Impacto

Acesso administrativo aos dados financeiros e à base de clientes de qualquer leitor que siga o comando.

****Condicoes:**** Nenhuma além de o leitor executar o comando publicado, que já expõe a porta 80.

Correcao sugerida

Substituir por geração de senha no ato e regenerar a lista list-operacao-industrial.

Criterios de aceite

- [] Correcao aplicada em `scripts/data/dossie-operacao-industrial.json`
- [] Teste automatizado que falha antes e passa depois
- [] Nova varredura nao reporta este achado
- [] Credencial rotacionada no provedor (o valor antigo esta comprometido)

--- FIM ISSUE 6 ---

--- ISSUE 7 ---

[Seguranca] Comando rápido do Odoo fixa a senha do Postgres em valor conhecido

****Labels:**** `security`, `media`

****Categoria:**** Chaves expostas

Problema

Senha de banco literal em comando copiável, materializada em output/02-dossies-verticais/vert-revops/materiais/vert-revops.html e em list-crm-erp-corporativo.html. O impacto é menor que os anteriores porque o Postgres fica atrás de '--link' e não é publicado diretamente, mas a senha vira conhecida para todo leitor e basta um segundo container na mesma rede ou uma exposição posterior da porta 5432 para valer.

Evidencia

`scripts/data/dossie-vertical-revops.json:41`

```

"comando\_rapido": "docker run -d -p 8069:8069 --name odoo-revops -e POSTGRES\_PASSWORD=\*\*\*\* --link db:db -t odoo:latest"

```

Impacto

Acesso ao banco do Odoo (dados de vendas, faturamento e clientes) por quem já tenha alcance à rede do container.

****Condicoes:**** Exige alcance à rede Docker do leitor ou publicação posterior da porta do Postgres.

Correcao sugerida

Gerar a senha no comando e regenerar vert-revops e list-crm-erp-corporativo.

Criterios de aceite

- [] Correcao aplicada em `scripts/data/dossie-vertical-revops.json`
- [] Teste automatizado que falha antes e passa depois
- [] Nova varredura nao reporta este achado
- [] Credencial rotacionada no provedor (o valor antigo esta comprometido)

--- FIM ISSUE 7 ---

--- ISSUE 8 ---

[Seguranca] Mesma senha de Postgres literal no dossiê de Salesforce

****Labels:**** `security`, `media`

****Categoria:**** Chaves expostas

Problema

Repetição do mesmo defeito em outro dossiê (materializado em output/02-dossies-verticais/vert-salesforce/materiais/vert-salesforce.html). Registrado à parte porque a correção do arquivo anterior não o alcança – os JSONs de dados são independentes.

Evidencia

`scripts/data/dossie-vertical-salesforce.json:163`

```

"comando\_rapido": "docker run -d -p 8069:8069 --name odoo -e POSTGRES\_PASSWORD=\*\*\*\* --link db:db -t odoo:latest"

```

Impacto

Idêntico ao achado de revops.

****Condições:**** Exige alcance à rede Docker do leitor.

Correcao sugerida

Gerar a senha no comando e regenerar vert-salesforce.

Criterios de aceite

- [] Correcao aplicada em `scripts/data/dossie-vertical-salesforce.json`
- [] Teste automatizado que falha antes e passa depois
- [] Nova varredura nao reporta este achado
- [] Credencial rotacionada no provedor (o valor antigo esta comprometido)

--- FIM ISSUE 8 ---

--- ISSUE 9 ---

[Seguranca] Credenciais operacionais vivas em .env de texto plano na estação de trabalho

****Labels:**** `security`, `media`

****Categoria:**** Chaves expostas

Problema

Verifiquei antes de classificar, sem presumir: 'git ls-files --error-unmatch .env' falha (não rastreado), 'git log --all -- .env' vem vazio, o pickaxe do valor do PAT retorna 0 commits, e .gitignore:19 o cobre. Não é vazamento: é superfície local. O que sobra de real é que o arquivo guarda em texto plano a senha de root da VPS, a senha do painel Portainer, um PAT do GitHub e a senha do Mautic, sem cifra e sem escopo – e qualquer processo, script ou subagente executado nesta pasta lê os quatro de uma vez (eu mesmo li). O grep pelos valores confirma que nenhum deles aparece em qualquer outro arquivo do repositório.

Evidencia

`.env:3, 8, 13, 18`

```

GITHUB\_TOKEN="\*\*\*\*"

PORTAINER\_PASSWORD="\*\*\*\*"

VPS\_PASSWORD="\*\*\*\*"

MAUTIC\_PASSWORD="\*\*\*\*"

```

Impacto

Um único processo hostil ou subagente mal direcionado nesta estação obtém root na VPS, controle do orquestrador de containers, escrita no repositório e a conta de e-mail marketing, sem precisar de escalada.

****Condicoes:**** Exige execução de código na estação de trabalho do dono. Sem vetor remoto.

Correcao sugerida

Nada a corrigir no repositório – o arquivo está corretamente ignorado. Decisão do dono: trocar a senha de root da VPS por chave SSH, reduzir o PAT do GitHub ao escopo mínimo com validade curta, e considerar um cofre em vez de texto plano. Rotação não é exigida por vazamento, e sim por higiene de credencial de longa duração.

Criterios de aceite

- [] Correcao aplicada em `.env`
- [] Teste automatizado que falha antes e passa depois
- [] Nova varredura nao reporta este achado
- [] Credencial rotacionada no provedor (o valor antigo esta comprometido)

--- FIM ISSUE 9 ---

--- ISSUE 10 ---

[Seguranca] Compilador de compêndio interpola JSON cru em HTML e em href, sem nenhum escape

****Labels:**** `security`, `media`

****Categoria:**** Inputs sem tratamento

Problema

O arquivo tem zero chamadas a `html.escape` (contei: 0) e monta todo o markup por f-string. O campo `repositorio_github` entra direto num atributo `href` delimitado por aspas duplas: um valor com aspas quebra o atributo e injeta manipulador de evento, e um valor `'javascript:...'` produz link executável. Os campos de texto (`nome`, `veredito`, `o_que_faz`) entram crus no corpo. A origem não é constante de código: os JSONs de `scripts/data` são preenchidos por pesquisa web da esteira (URLs de repositório, descrições e títulos coletados de terceiros) e o arquivo de dados é escolhido por argumento de CLI.

Evidencia

```
`scripts/compilar_compendio_diamante.py:329, 406, 358, 383, 402, 418, 445, 504`
```

```
...
```

```
repo_url = f.get("repositorio_github", "#")
```

```
...
```

```
<a class="repo-btn" href="{repo_url}" rel="noopener noreferrer" target="_blank">
```

```
...
```

Impacto

XSS armazenado no HTML distribuído ao leitor final – execução de script no contexto do documento aberto por advogados, médicos e gestores, que é o público declarado.

****Condicoes:**** Exige que um campo do JSON de dados carregue conteúdo hostil, o que é plausível porque esses campos vêm de coleta web.

Correcao sugerida

Adicionar helper `_esc` com `html.escape(..., quote=True)` aplicado a toda interpolação de campo do JSON, e validar `repositorio_github` contra allowlist de esquema (`^https?://`).

Criterios de aceite

- [] Correcao aplicada em ``scripts/compilar_compendio_diamante.py``
- [] Teste automatizado que falha antes e passa depois
- [] Nova varredura nao reporta este achado

--- FIM ISSUE 10 ---

```
--- ISSUE 11 ---  
## [Seguranca] Compilador de dossiê vertical repete a interpolação sem escape em href e corpo  
  
**Labels:** `security`, `media`  
**Categoria:** Inputs sem tratamento  
  
### Problema  
  
Mesmo padrão do compilador de compêndio, também com zero chamadas de escape no arquivo inteiro.  
Consome scripts/data/dossie-vertical-*.json, cujos campos de URL e descrição vêm de coleta  
externa.  
  
### Evidencia  
  
`scripts/compilar_compendio_vertical.py:388, 380, 384`  
  
`...`  
`<a class="repo-btn" href="{repo_url}" rel="noopener noreferrer" target="_blank">`  
`...`  
  
### Impacto  
  
XSS armazenado nos dossiês verticais distribuídos.  
  
**Condições:** Campo hostil no JSON de dossiê.  
  
### Correção sugerida  
  
Mesmo helper de escape e mesma allowlist de esquema de URL.  
  
### Critérios de aceite  
  
- [ ] Correção aplicada em `scripts/compilar_compendio_vertical.py`  
- [ ] Teste automatizado que falha antes e passa depois  
- [ ] Nova varredura não reporta este achado  
  
--- FIM ISSUE 11 ---
```

--- ISSUE 12 ---

[Seguranca] Compilador de ecossistema monta href com aspas simples, tornando a quebra de atributo trivial

****Labels:**** `security`, `media`

****Categoria:**** Inputs sem tratamento

Problema

Zero chamadas de escape no arquivo. A linha 2450 é a pior das três porque o atributo é delimitado por aspas simples: basta um caractere ' no valor para fechar o atributo e injetar onmouseover=. Na mesma linha, nome, saas_substituido_direto e economia_anual_str também entram crus. As linhas 902 e 1007 repetem o href sem escape com aspas duplas.

Evidencia

`scripts/compilar\_ecossistema\_tripartite.py:902, 1007, 2450`

...

```
linhas_tab += f"<tr>...<td><a href='{f_item['repositorio_github']}' target='_blank'
style='...'>GitHub</a></td></tr>"
```

...

Impacto

XSS armazenado nos dossiês de macro-ecossistema.

****Condições:**** Campo hostil no JSON de ecossistema.

Correção sugerida

Escapar com quote=True e padronizar aspas duplas nos atributos.

Critérios de aceite

- [] Correção aplicada em `scripts/compilar\_ecossistema\_tripartite.py`
- [] Teste automatizado que falha antes e passa depois
- [] Nova varredura não reporta este achado

--- FIM ISSUE 12 ---

--- ISSUE 13 ---

[Seguranca] Bibliografia de trilha injeta URL, título e autor de pesquisa web direto no HTML

****Labels:**** `security`, `media`

****Categoria:**** Inputs sem tratamento

Problema

Zero chamadas de escape no arquivo. Este é o caso de origem externa mais direto do repositório: rastreei scripts/gerar_trilha_aprendizado.py:83-89, que lê scripts/data/trilha-<slug>.json e repassa o dicionário sem tratamento; inspecionei scripts/data/trilha-screenpipe.json e os registros de recurso são fontes web reais (url https://dev.to/..., título, autor, aprendizado_chave em texto livre). Ou seja, o campo que vai para o href é exatamente uma URL de terceiro coletada pela esteira, e o esquema não é validado – 'javascript:' passa.

Evidencia

`scripts/padroes/template\_trilha\_aprendizado.py:242, 249, 251, 252, 254, 257, 258, 259`

...

{r['título']}

...

Impacto

XSS armazenado nas trilhas de aprendizado entregues ao leitor final.

****Condições:**** Exige que a coleta de fontes traga um título, autor ou URL com conteúdo hostil.

Correcao sugerida

Escapar todos os campos de recurso com quote=True e rejeitar URL cujo esquema não seja http/https.

Criterios de aceite

- [] Correcao aplicada em `scripts/padroes/template\_trilha\_aprendizado.py`
- [] Teste automatizado que falha antes e passa depois
- [] Nova varredura nao reporta este achado

--- FIM ISSUE 13 ---

--- ISSUE 14 ---

[Seguranca] Tabela de fontes verificadas do manual operacional interpola dados de terceiros sem escape

Labels: `security`, `media`

Categoria: Inputs sem tratamento

Problema

Zero chamadas de escape. Rastreei a origem em scripts/gerar_manual_operacional.py:217-230: os registros vêm de scripts/data/manual-<slug>.json, seção de fontes verídicas, que a própria documentação do Fluxo 3 descreve como 'auditoria de 5 fontes verificadas' coletadas da web. Título, autor e URL são conteúdo de terceiro entrando em corpo e em href sem tratamento.

Evidencia

`scripts/padroes/template\_manual\_operacional.py:426, 427, 428, 429, 430`

...

```
<td>{r['autor_ou_canal']}
```

```
<td><a href="{r['url']}" target="_blank" rel="noopener noreferrer" style="...">Acessar Fonte Original</a></td>
```

...

Impacto

XSS armazenado nos manuais operacionais de VPS, que são o material com maior circulação do projeto.

Condições: Fonte coletada com conteúdo hostil.

Correção sugerida

Escapar os campos de fonte com quote=True e validar o esquema da URL.

Critérios de aceite

- [] Correção aplicada em `scripts/padroes/template\_manual\_operacional.py`
- [] Teste automatizado que falha antes e passa depois
- [] Nova varredura não reporta este achado

--- FIM ISSUE 14 ---

--- ISSUE 15 ---

[Seguranca] Campo SMTP_PASSWORD escapa do padrão de placeholder que o próprio manual ensina a substituir

****Labels:**** `security`, `baixa`

****Categoria:**** Chaves expostas

Problema

No mesmo bloco, DB_PASSWORD e APP_SECRET usam a forma <COLE-AQUI-...> e o texto seguinte instrui: 'Repare que no arquivo acima existem campos escritos assim: <COLE-AQUI-...>. Eles nao sao senhas de verdade.' O SMTP_PASSWORD não segue esse padrão e carrega um literal, então a instrução não o cobre. Um leitor não-técnico seguindo o manual à risca não é levado a trocá-lo. O valor em si não autentica em lugar nenhum, então não há credencial exposta – o defeito é a inconsistência que ensina o leitor a confiar num critério visual incompleto.

Evidencia

`scripts/data/manual-mautic.json:163`

...

SMTP_USER=seu-email@seuservidor.com.br\nSMTP_PASSWORD=****\nSMTP_ENCRYPTION=tls

...

Impacto

Configuração de e-mail quebrada e, pior, erosão da regra de leitura que o próprio manual estabelece para identificar o que precisa ser substituído.

****Condicoes:**** Depende do leitor seguir literalmente o critério <COLE-AQUI-...> que o manual define.

Correcao sugerida

Trocar por <COLE-AQUI-A-SUA-SENHA-DO-EMAIL> para uniformizar o padrão, e regenerar o manual do Mautic.

Criterios de aceite

- [] Correcao aplicada em `scripts/data/manual-mautic.json`
- [] Teste automatizado que falha antes e passa depois
- [] Nova varredura nao reporta este achado
- [] Credencial rotacionada no provedor (o valor antigo esta comprometido)

--- FIM ISSUE 15 ---

--- ISSUE 16 ---

[Seguranca] Sanitização parcial: vps_generator escapa a tabela e esquece o target-box

Labels: `security`, `baixa`

Categoria: Inputs sem tratamento

Problema

O arquivo define o helper correto `_esc` (linhas 6-15, com `html.escape quote=True`) e o aplica em nove pontos (1142, 1147, 1159, 1163, 1167, 1174, 1189, 1194), mas não nestes dois. É a assinatura clássica de sanitização parcial: o mesmo valor passa escapado quando entra pela tabela markdown e cru quando entra pelo target-box. A origem é o slug de CLI:

`scripts/modulos/vps_decision_engine.py:319-336` devolve `name=f'Ferramenta {tool_slug}'` para qualquer slug não cadastrado – que é o caminho normal de qualquer alvo novo – propagando o argumento de `--ferramenta` até o `<h4>`.

Evidencia

``scripts/modulos/vps_generator.py:1227, 1228, 1231, 1232``

...

`target_name = prof.get('name', hero_title)`

`role_name = prof.get('role', 'Stack e Módulos de Produção')`

...

`<h4>{target_name}</h4>`

`<p>{role_name} · Garantia de Risco Zero...</p>`

...

Impacto

Injeção de markup no HTML de auditoria VPS. Explorabilidade contida porque quem passa o slug é o operador ou o agente que ele dirige, não um terceiro remoto.

Condições: Requer que o slug chegue de fonte não confiável – plausível neste projeto, onde os fluxos são acionados por comandos de agente (`'/fluxo5 [slug]'`) sujeitos a injeção de prompt, mas não é vetor direto.

Correção sugerida

Envolver `target_name` e `role_name` no `_esc` que já existe no arquivo. Correção de duas linhas.

Critérios de aceite

- [] Correção aplicada em ``scripts/modulos/vps_generator.py``
- [] Teste automatizado que falha antes e passa depois
- [] Nova varredura não reporta este achado

--- FIM ISSUE 16 ---

--- ISSUE 17 ---

[Seguranca] Renderizador desfaz o escape do Pandoc em blocos Mermaid e ainda liga securityLevel loose

Labels: `security`, `baixa`

Categoria: Inputs sem tratamento

Problema

O Pandoc já havia escapado o bloco de código; este replace reverte a sanitização e reinsere < e > crus no documento, de modo que um bloco `mermaid` contendo `</pre><script>` escapa do `<pre>`. securityLevel 'loose' agrava: o Mermaid passa a aceitar HTML em labels e callbacks click. A origem é o conteúdo de arquivos .md lidos do disco (verifiquei scripts/compilar_playbook_fluxo1.py:53-65, que concatena todos os .md de especificacoes/), hoje escritos pelo próprio projeto – por isso baixa, e não média.

Evidencia

```
`scripts/renderizador_html_diamante.py:176, 177, 178, 179, 199, 200, 201, 202, 203`
```

```
...
```

```
conteudo = conteudo.replace("&", "&").replace(""", "'').replace("<", "<")
    .replace(">", ">")
return f'<pre class="mermaid">\n{conteudo.strip()}\n</pre>'
```

```
...
```

```
mermaid.initialize({{ startOnLoad: true, theme: 'neutral', securityLevel: 'loose' }});
```

```
...
```

Impacto

Execução de script no playbook HTML se um .md de especificação vier de fora ou for editado por terceiro.

Condições: Depende de um .md de origem não confiável entrar em especificacoes/. Hoje todos são do próprio autor.

Correcao sugerida

Trocar securityLevel para 'strict' e, em vez de desescapar o bloco inteiro, reverter só as entidades dentro do texto do diagrama após validar que ele não contém '<'.

Criteria de aceite

- [] Correcao aplicada em `scripts/renderizador\_html\_diamante.py`
- [] Teste automatizado que falha antes e passa depois
- [] Nova varredura nao reporta este achado

--- FIM ISSUE 17 ---

--- ISSUE 18 ---

[Seguranca] linter.js executa eslint via shell com lista de arquivos concatenada sem quoting

****Labels:**** `security`, `baixa`

****Categoria:**** Inputs sem tratamento

Problema

execSync recebendo string roda através do shell. globs vem de process.argv ou de stdin separado por linha, sem quoting nem escape, então um caminho contendo ;, &&, crase ou \$() executa comando arbitrário. Verifiquei os chamadores: nenhum script do repositório invoca linter.js – as únicas referências são documentação em relatórios. Sem alimentador automatizado, o vetor depende de alguém canalizar uma lista de nomes de arquivo de origem não confiável para o stdin.

Evidencia

`scripts/linter.js:24, 29, 32`

...

```
const args = ["eslint", "--format=json", fix ? "--fix" : "", ...globs].filter(Boolean);
try {
  const output = execSync(args.join(" "), {
  ...
```

Impacto

Execução de comando arbitrário na estação de quem rodar o linter com entrada hostil.

****Condicoes:**** Exige alimentar o stdin com nomes de arquivo controlados por terceiro. Não há pipeline atual que faça isso.

Correcao sugerida

Trocar por execFileSync("eslint", args), que não passa pelo shell.

Criterios de aceite

- [] Correcao aplicada em `scripts/linter.js`
- [] Teste automatizado que falha antes e passa depois
- [] Nova varredura nao reporta este achado

--- FIM ISSUE 18 ---

--- ISSUE 19 ---

[Segurança] Slug de CLI vira caminho de escrita sem allowlist, e componente absoluto descarta a base

****Labels:**** `security`, `baixa`

****Categoria:**** Inputs sem tratamento

Problema

t_slug vem de --ecossistema/--ferramenta/--ecossistemas/--ferramentas sem normalização nem allowlist, e scripts/modulos/vps_decision_engine.py:319-336 aceita slug desconhecido em vez de recusar. Duas propriedades combinam: os.path.join descarta a base quando o último componente é absoluto, então --ferramenta 'C:\Windows\Temp\x' escreve fora de output/; e '../..' também escapa. O destino é criado em vps_generator.py:184 e recebe cerca de vinte open(...,'w') de nomes fixos.

Evidencia

`scripts/run\_fluxo5.py:109, 110, 111, 112, 113, 114, 116`

...

out_dir = os.path.join(root_vps_output, "ecossistemas", t_slug)

...

out_dir = os.path.join(root_vps_output, "ferramentas", t_slug)

gen = VPSGenerator(audit_data, decision, out_dir, base_domain=args.base_domain)

...

Impacto

Criação de diretório e sobrescrita de arquivo fora da raiz do projeto, limitada aos nomes de arquivo fixos do gerador.

****Condições:**** Quem passa o slug é o operador, que já pode escrever onde quiser – não há fronteira de privilégio cruzada. Relevante apenas porque os fluxos são acionados por agente ('/fluxo5 [slug]'), o que abre a possibilidade de o valor vir de injeção de prompt.

Correção sugerida

Validar o slug contra `^[a-z0-9-]{1,35}$` (a própria regra R13 já exige isso) e recusar o resto, em vez de cair em perfil genérico.

Critérios de aceite

- [] Correção aplicada em `scripts/run\_fluxo5.py`
- [] Teste automatizado que falha antes e passa depois
- [] Nova varredura não reporta este achado

--- FIM ISSUE 19 ---

```
--- ISSUE 20 ---
## [Seguranca] Slug do Fluxo 1 entra em caminho sem remover separadores nem sequência de subida

**Labels:** `security`, `baixa`
**Categoria:** Inputs sem tratamento

### Problema

A 'limpeza' remove apenas três literais e não filtra / \ nem '..', então --slug
'../../../../tmp/x' é segmentado pelo pathlib e escapa de output/, com mkdir(parents=True) criando
a árvore e write_text gravando. As linhas 136-138 usam o mesmo argumento como caminho de leitura
direto, inclusive absoluto, permitindo carregar JSON de fora da raiz.

### Evidencia

`scripts/compilar_lista_horizontal_tripartite.py:131, 136, 137, 138, 152, 153, 154, 155, 156`

...
slug_limpo = slug_ou_json.replace("list-", "").replace(".html", "").replace(".json", "")
...
out_bundle = BASE_DIR / "output" / "01-listas-horizontais" / f"list-{slug_limpo}"
out_materiais.mkdir(parents=True, exist_ok=True)
...

### Impacto

Escrita e leitura de arquivo fora do diretório do projeto a partir de um argumento de CLI.

**Condições:** Mesma fronteira do achado anterior: o operador, ou o agente que ele dirige,
controla o argumento.

### Correção sugerida

Aplicar a validação de slug da regra R13 antes de compor qualquer caminho e resolver o resultado
com Path.resolve(), recusando o que não estiver sob BASE_DIR.

### Critérios de aceite

- [ ] Correção aplicada em `scripts/compilar_lista_horizontal_tripartite.py`
- [ ] Teste automatizado que falha antes e passa depois
- [ ] Nova varredura não reporta este achado

--- FIM ISSUE 20 ---
```